AI Incident Report Assistant

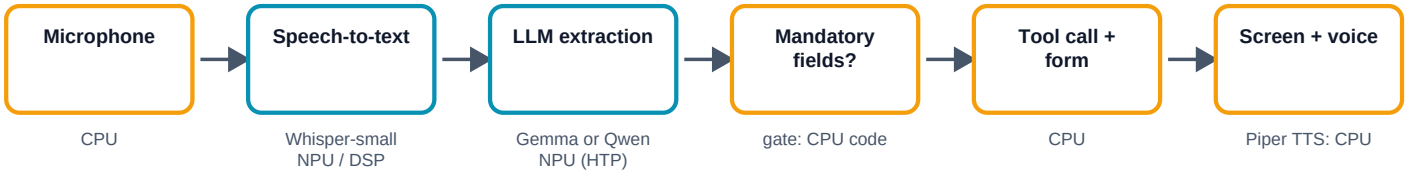
Final report: voice-driven incident reporting that runs entirely on a Qualcomm HDK8650 (Snapdragon 8 Gen 3, Hexagon NPU)

1. Summary

A person describes an incident in natural speech. The phone transcribes it, extracts the ten form fields, checks the four mandatory ones and either asks **one** clarifying question or fills the form and speaks a confirmation. The user then reviews, edits and submits the report. **Everything runs on the phone with no network and no laptop**: speech recognition on the Hexagon DSP, the language model on the NPU, spoken replies from an offline voice engine. No cloud API is called at run time (models were downloaded once, beforehand, on a laptop).

Result (measured on the board, Wi-Fi off)	Value
Speech recognition (Whisper-small, encoder and decoder both on the DSP)	4.4% word error rate on 10 natural narrations; 4.0% with 10 dB noise
Information extraction and field mapping (22 narrations, 2 test sets)	97 of 99 fields correct; 22 of 22 correct decisions on whether to ask or finish
Time for one spoken report	about 2 s transcription + 14 s extraction (4 s in the optional quick mode)
Encoder speed (same model, same clip)	167 ms on the NPU, 316 ms on the GPU, 2168 ms on the CPU (13x)
Hardware evidence	NPU message-channel interrupts 0/s idle, 40-60/s while working; audio DSP flat

2. Pipeline, as specified in the problem statement



Cyan = runs on the NPU/DSP; amber = ordinary CPU code. The mel spectrogram, tokenizer, the deterministic mandatory-field gate, the form and the UI are CPU code, as in Qualcomm's own reference apps.

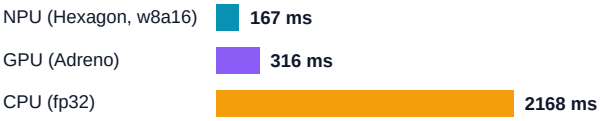
Slide stage	Implementation	Details
Speech-to-text	Whisper-small, encoder w8a16 and decoder fp16 as SNPE graphs prepared for HTP v75	Built here from OpenAI Whisper; encoder cosine 0.998 vs fp32, decoder 0.99999 vs PyTorch on the board
LLM information extraction	Gemma 4 E4B Q4_0 via GenieX (llama.cpp Hexagon backend); greedy decoding; JSON grammar	Output limited to the 10 contract fields; same prompt always gives the same answer
Mandatory fields present?	Kotlin port of the agent brain (verified against the Python original), contract.json unchanged	Missing field: ask one question. All four present: populate_incident_form tool call
Populate form / review / submit	Native touch UI: Edit, Clear, severity chips; Submit saves the 10 fields as JSON on the phone	Edits pass through the same mandatory-field gate
Screen and text-to-speech	Live status, step-by-step processing, spoken replies from an offline Piper voice (sherpa-onnx engine)	The voice engine declares no network permission

3. Models and where each part runs

Model	Format	Share of weights on the NPU	Role / why chosen
Whisper-small	SNPE DLC: encoder w8a16 (187 MB), decoder fp16 (618 MB)	100% (both graphs)	Clear gain over Whisper-tiny under noise (WER 12.7% vs 18.3% at 5 dB) for +1 s
Gemma 4 E4B (default)	Q4_0 GGUF, 5.15 GB	40% (2171 of 5426 MiB); the rest are large embedding tables	Most careful at not inventing details; the model suggested in the documentation
Qwen3-1.7B (quick mode)	all-Q4_0 GGUF, 0.97 GB (re-quantised here)	69% (82% of unique weights); all 28 layers	3x faster, a third of the CPU time; needs an output guard
Whisper-tiny	DLC encoder + TFLite decoder	encoder 100%	First working pipeline; automatic fallback
Piper en_US-lessac	ONNX VITS voice, 87 MB	0% (CPU)	Spoken replies; the slide requires text-to-speech output

Honest reading: all AI compute for speech recognition runs on the NPU. Gemma is a mixed model: about 40% of its weight bytes are on the NPU and its large lookup tables and some tensors stay in CPU memory, so token generation is only 1.1x faster than on the CPU (prompt processing is 6.4x faster). The Hexagon backend only runs Q4_0 tensors, which is why the all-Q4_0 Qwen file reaches 69%.

4. Evidence that it really runs on the NPU



- **Hardware counter:** the interrupt counter of the CPU-to-cDSP message channel is 0/s idle and 40-60/s during speech recognition and extraction; the audio-DSP counter stays at 0.
- **NPU temperature:** matrix and vector units rise from about 55 C to 92-95 C under load.
- **Placement log:** GenieX reports where weights were placed; the app refuses to claim the NPU otherwise.
- **Live in the app:** a small strip shows this counter, the temperature and which model is working.

Whisper-small encoder, one 30 s clip (snpe-net-run on the board).

5. The five evaluation criteria

Criterion	How it is met	Evidence
All AI on-device with Snapdragon acceleration, no cloud	Whisper on the DSP, LLM on the NPU; no outbound network code; voice engine has no network permission; tested with Wi-Fi off	Counters above; INTERNET permission is used only for an optional loopback link to the laptop UI
Natural conversational speech, not commands	Free speech goes to Whisper then an LLM; no command grammar or trigger words	Test narrations include hesitations, chatter, self-correction, indirect wording
Speech recognition accuracy	Whisper-small on the DSP with noise-floor tracking and speech trimming	WER 4.4% (clean) and 4.0% (10 dB noise); the smaller Whisper-tiny scored 12.7% to 19.7% on the same noisy sentences
Extraction accuracy	Grammar-constrained, greedy LLM output; placeholders removed	97/99 fields on 22 narrations; invented values: 2 (Severity guessed; optional)
Correct mapping to report fields	Ten contract keys only; the four mandatory fields gate completion	22/22 correct ask-or-complete decisions, including four reports that must NOT be completed

6. Accuracy and speed: both language models

	Gemma 4 E4B (default)	Qwen3-1.7B all-Q4_0 + guard (quick)
Original 10 narrations, typed	47/48 fields, gate 10/10	46/48 fields, gate 10/10
12 new held-out narrations, typed	50/51 fields, gate 12/12	51/51 fields, gate 12/12
Speech through Whisper: clean / 10 dB noise	44/48 / 45/48	43/48 / 43/48
3-turn clarification dialogue	pass	pass
Time per extraction (4 CPU threads)	13.9 s	4.2 s
CPU time per extraction	52 cpu-s	15 cpu-s
Memory in use (app)	3.75 GB, about 2.3 GB free	about 0.65 GB resident

The app starts on Gemma; a **Model** toggle switches to Qwen in a few seconds and back. Without its output guard Qwen completed 2 of 12 held-out reports that should have asked a question (it wrote "current time" as the date); the guard removes such placeholder values and invented follow-up advice, and is applied in quick mode only. Test narrations were spoken by a synthetic voice, so the speech numbers are a proxy for real speakers.

7. Engineering decisions that mattered

- **NPU only, never silently CPU.** The reference sample listed {DSP, CPU} and fell back quietly; the app now refuses to report NPU use unless placement is verified.
- **Whisper decoder on the DSP.** Two bugs found by comparing against PyTorch: the converter transposes 3-D inputs, and an fp16 graph cannot hold token ids above 2048 (50258 became 50272), so embeddings are looked up on the CPU.
- **Truly repeatable extraction.** Temperature 0 still left sampling on; top-k 1 with a fixed seed makes the same prompt give the same answer.
- **CPU threads 8 to 4.** The SDK's eight busy-polling threads cost 98 CPU-seconds per extraction; four give the same speed for 52.

- **No frozen screen.** The UI waited on a lock held for the whole 14 s model call; state is now published as lock-free snapshots.
- **One verified brain.** The Kotlin port reproduces backend/agent_brain.py exactly (golden-file tests), so the locked contract and brain were never modified.

8. Limitations and next steps

Limitation	Mitigation / next step
Names and numbers are the weak point of speech recognition ("Ravi" heard as "Rovey", "Room 2" as "room to")	Review screen lets the user correct any field; next: spoken read-back of low-confidence words
About 14 s of waiting after speech with Gemma; model also guesses Severity when it was not said	Quick Qwen mode (4 s); Severity is optional and editable
In-progress report is only in memory; reports are local JSON; English only; one microphone distance (about a metre)	Draft autosave, headset or push-to-talk, more languages, photo attachment, automatic time and place
Test set is 22 narrations with a synthetic voice; the board clock was days behind	Next: tests with real speakers on site; set the clock

9. What would matter most in a real incident (not implemented)

Feature	Why it matters when someone is stressed, hurried or offline
Spoken read-back with yes/no for low-confidence names, numbers and rooms	A wrong room or name is the most damaging error; Whisper exposes token probabilities, so doubtful words can be repeated aloud and confirmed hands-free
Emergency fast path: one button, save immediately, mark critical, ask follow-ups afterwards	The four-field gate must not delay the very first record of a life-threatening event
Draft autosave, crash recovery and an outbox that syncs only if a network ever appears	A killed app or empty battery must not lose a report; AI stays on-device
Automatic time and place, photo or audio evidence, more languages, headset push-to-talk	Fewer things to say, better evidence, and usable by reporters who are not comfortable in English or cannot hold the phone
Safety checklist by incident type; audit trail with a hash; PIN lock; thermal and battery guard	Reminds the reporter of immediate actions, proves the AI step ran on the device, protects names and injuries, avoids a throttled device

10. Reproduce

Code: **github.com/nishulucyna/B612P**, branch **main**. Phone only: run `scripts\install_standalone.ps1` once over USB, unplug, open **Incident Assistant**, wait about 40 s for the green chip, tap the microphone. Evidence and tests: `scripts/npu_proof.py` (hardware trace, saved as deliverables/npu_proof.html), `scripts/e2e_eval.py` (accuracy), `python -m pytest`. Large model files are not in the repository.